

Week3_예습과제_진예원

5장 합성곱 신경망 I

5.1 합성곱 신경망(Convolutional Neural Network, CNN 또는 ConvNet)

5.1.1 합성곱층의 필요성

5.1.2 합성곱 신경망 구조

5.1.3 1D, 2D, 3D 합성곱

5.2 합성곱 신경망 맛보기

5장 합성곱 신경망 I

5.1 합성곱 신경망(Convolutional Neural Network, CNN 또는 ConvNet)

- 음성 인식이나 이미지/영상 인식에 주로 사용
- 다차원 배열 데이터를 처리하도록 구성 → 다차원 배열 처리에 특화(ex. 컬러 이미지)
- 이미지 전체를 한 번에 계산 X. 이미지의 국소적 부분 계산 → 시간, 자원 절약 + 이미지 세밀한 부분까지 분석 가능

5.1.1 합성곱층의 필요성

- 이미지 분석은 배열을 펼쳐서(flattening) 각 픽셀에 가중치를 곱하여 은닉층으로 전달
 - 펼쳐서 분석하면 데이터의 공간적 구조 무시
 - ⇒ 방지 목적으로 도입된 것이 합성곱

5.1.2 합성곱 신경망 구조

1. 입력층(input layer)

- 입력 이미지 데이터가 최초로 거치게 되는 계층
- 이미지 - 높이, 너비, 채널 3차원 데이터
 - 채널
 - 그레이스케일(gray scale) → 1
 - 컬러(RGB) → 3

- ex) 그림 예시 이미지 형태(shape) - (4, 4, 3)

2. 합성곱층(convolutional layer)

- 입력 데이터에서 특성을 추출하는 역할
- 특성 추출 진행 과정 - 입력 이미지가 들어왔을 때 이미지에 대한 특성을 감지 위해 커널(kernel)이나 필터 사용. 커널/필터는 이미지의 모든 영역 훑으면서 특성 추출
⇒ 추출될 결과물 : 특성 맵(feature map)
 - 커널 보통 3X3, 5X5 크기로 적용
 - 스트라이드(stride)에 따라 순차적 이동하며 모든 입력값과의 합성곱 연산으로 새로운 특성 맵 생성
(스트라이드: 필터를 적용하는 위치와 간격)
- 그레이 스케일 이미지 크기 (6, 6, 1), 스트라이드 1일 때 이동하는 과정
 1. 입력 이미지에 3 X 3 필터 적용 - 입력 이미지와 필터 포개 놓고 대응되는 숫자 끼리 곱한 후 모두 더함
 - $(1 \times 1) + (0 \times 0) + (0 \times 1) + (0 \times 0) + (1 \times 1) + (0 \times 0) + (0 \times 1) + (0 \times 0) + (1 \times 1) = 3$
 2. 필터가 1만큼 이동
 - $(0 \times 1) + (0 \times 0) + (0 \times 1) + (1 \times 0) + (0 \times 1) + (0 \times 0) + (0 \times 1) + (1 \times 0) + (1 \times 1) = 1$
 3. 필터가 1만큼 두 번째 이동
 - $(0 \times 1) + (0 \times 0) + (0 \times 1) + (0 \times 0) + (0 \times 1) + (1 \times 0) + (1 \times 1) + (1 \times 0) + (0 \times 1) = 1$
 4. 필터가 1만큼 세 번째 이동
 - $(0 \times 1) + (0 \times 0) + (1 \times 1) + (0 \times 0) + (1 \times 1) + (0 \times 0) + (1 \times 1) + (0 \times 0) + (0 \times 1) = 3$
 5. 필터가 1만큼 네 번째 이동
 - $(0 \times 1) + (1 \times 0) + (0 \times 1) + (0 \times 0) + (0 \times 1) + (1 \times 0) + (1 \times 1) + (0 \times 0) + (0 \times 1) = 1$
 6. 필터가 1만큼 마지막으로 이동
 - $(0 \times 1) + (1 \times 0) + (0 \times 1) + (0 \times 0) + (1 \times 1) + (0 \times 0) + (0 \times 1) + (1 \times 0) + (0 \times 1) = 1$
- 이미지 크기 원본 (6, 6, 1) → 특성 맵 크기 (4, 4, 1)

- 컬러 이미지의 합성곱
 - 필터 채널 3(필터 개수는 한 개)
 - RGB 각각에 서로 다른 가중치로 합성곱 적용 후 결과 더함
- 필터가 두 개 이상인 합성곱 - 필터 각각은 특성 추출 결과의 채널이 됨
- 요약
 - 입력 데이터: $W_1 \times H_1 \times D_1$ (W_1 : 가로, H_1 : 세로, D_1 : 채널 또는 깊이)
 - 하이퍼파라미터
 - 필터 개수: K
 - 필터 크기: F
 - 스트라이드: S
 - 패딩: P
 - 출력 데이터
 - $W_2 = (W_1 - F + 2P) / S + 1$
 - $H_2 = (H_1 - F + 2P) / S + 1$
 - $D_2 = K$

3. 풀링층(pooling layer)

- 특성 맵의 차원을 다운 샘플링하여 연산량 감소 & 주요한 특성 벡터 추출 → 학습 효과적으로
- 다운 샘플링: 이미지 축소
- 풀링 연산 요소
 1. 최대 풀링(max pooling): 대상 영역에서 최댓값 추출
 - 대부분 합성곱 신경망에서 사용
 2. 평균 풀링(average pooling): 대상 영역에서 평균 반환
 - 중요한 가중치 갖는 값의 특성 희미해질 수 있음
- 최대 풀링 연산 과정
 1. 첫 번째 최대 풀링 과정
 2. 두 번째 최대 풀링 과정

3. 세 번째 최대 풀링 과정

4. 네 번째 최대 풀링 과정

◦ 최대 풀링과 평균 풀링 결과 비교

- 요약

- 입력 데이터: $W_1 X H_1 X D_1$

- 하이퍼파라미터

- 필터 크기: F

- 스트라이드: S

- 출력 데이터

- $W_2 = (W_1 - F)/S + 1$

- $H_2 = (H_1 - f)/S + 1$

- $D_2 = D_1$

4. 완전 연결층(fully connected layer)

- 합성곱층과 풀링층 거치며 차원 축소된 특성 맵이 최종적으로 전달되는 층
- 이미지 펼쳐짐(flatten) - 3차원 벡터 → 1차원 벡터

5. 출력층(output layer)

- 마지막 출력층의 소프트맥스 활성화 함수 사용됨
- 입력받은 값을 0 ~ 1 사이의 값으로 출력
- 이미지가 각 레이블(label)에 속할 확률 값이 출력. 이때 가장 높은 확률 값을 갖는 레이블 = 최종 값으로 선정

5.1.3 1D, 2D, 3D 합성곱

- 합성곱은 이동하는 방향의 수와 출력 형태에 따라 1D, 2D, 3D로 분류

1. 1D 합성곱

- 필터가 시간을 축으로 좌우로만 이동할 수 있는 합성곱
- 입력(W)과 필터(k)에 대한 출력 = W
- ex. 입력 [1, 1, 1, 1, 1], 필터 [0.25, 0.5, 0.25] → 출력 [1, 1, 1]
- 출력 형태 = 1D의 배열

- 그래프 곡선을 완화할 때 주로 사용

2. 2D 합성곱

- 필터가 다음 그림과 같이 방향 두 개로 움직이는 형태
- 입력(W, H)과 필터(k, k)에 대한 출력 = (W, H)
- 출력 형태 2D 행렬

3. 3D 합성곱

- 필터가 다음 그림과 같이 방향 세 개로 움직이는 형태
- 입력(W, H, L)과 필터(k, k, d)에 대한 출력 = (W, H, L)
- 출력 형태 3D
- $d < L$ 유지가 중요

4. 3D 입력을 갖는 2D 합성곱

- 입력이 3D 형태임에도 출력 형태가 2D 행렬 취하는 것
- 필터에 대한 길이(L)가 입력 채널의 길이(L)와 같아야 하기 때문
- 입력(W, H, L)에 필터(k, k, L)를 적용 → 출력(W, H)
- 필터는 다음 그림과 같이 방향 두 개로 움직이는 형태
- ex. LeNet-5, VGG → 6장

5. 1X1 합성곱

- 3D 형태로 입력
- 입력(W, H, L)에 필터(1, 1, 1)를 적용 → 출력(W, H)
- 채널 수를 조정해서 연산량 감소되는 효과
- ex. GoogLeNet → 6장

5.2 합성곱 신경망 맛보기

fashion_mnist 데이터셋

- 토치비전(torchvision)에 내장된 예제 데이터. 운동화, 셔츠와 같은 작은 이미지의 모음
- 기본 MNIST 데이터셋처럼 열 가지로 분류될 수 있는 28X28 픽셀의 이미지 7만 개로 구성되어 있다
- 훈련 데이터(train_images) - 0에서 255 사이의 값을 갖는 28X28 크기의 넘파이 배열

- 레이블(정답) 데이터(train_labels) - 0에서 9까지 정수 값을 갖는 배열. 0에서 9까지 정수 값은 이미지의 클래스를 나타내는 레이블
- 레이블과 클래스
0 : T-Shirt, 1 : Trouser, 2 : Pullover, 3 : Dress, 4 : Coat, 5 : Sandal, 6 : Shirt, 7 : Sneaker, 8 : Bag, 9 : Ankle Boot

라이브러리 호출

```
import numpy as np
import matplotlib.pyplot as plt

import torch
import torch.nn as nn
from torch.autograd import Variable
import torch.nn.functional as F

import torchvision
import torchvision.transforms as transforms #데이터 전처리 위
해 사용
from torch.utils.data import Dataset, DataLoader
```

- 파이토치 GPU 권장 But GPU 없어도 정상적으로 실행 가능

CPU 혹은 GPU 장치 확인

```
device = torch.device("cuda:0" if torch.cuda.is_available()
else "cpu")
```

GPU 사용

```
#하나의 GPU 사용
device = torch.device("cuda:0" if torch.cuda.is_available()
else "cpu")
model = Net()
model.to(device)

#다수의 GPU 사용 - nn.DataParallel
```

```

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = Net()
if torch.cuda.device_count() > 1:
    model = nn.DataParallel(net)
    model.to(device)

```

- nn.DataParallel 사용할 경우 - 배치 크기(batch size)가 알아서 각 GPU로 분배됨
→ GPU 수만큼 배치 크기도 늘려줘야함

fashion_mnist 데이터셋 내려받기

```

#훈련 데이터셋 내려받음
train_dataset = torchvision.datasets.FashionMNIST("./data",
download=True,
transform=
transforms.ToTensor())
#테스트 데이터셋 내려받음
test_dataset = torchvision.datasets.FashionMNIST("./data",
download=True,
train=False, transform=transforms.Compose([transforms.ToTensor()]))

```

- torchvision.datasets = torch.utils.data.Dataset의 하위클래스. 다양한 데이터셋 (CIFAR, COCO, MNIST, ImageNet 등) 포함
- torchvision.datasets에서 사용하는 주요한 파라미터
 - 첫 번째 파라미터: FashionMNIST를 내려받을 위치 지정
 - download: download를 True로 변경 → 첫 번째 파라미터의 위치에 해당 데이터셋 있는지 확인 후 내려받음
 - transform: 이미지를 텐서(0~1)로 변경
- 내려받은 fashion_mnist 데이터셋 확인하려면 ./data 폴더 하위에서 FashionMNIST 폴더 확인하면 됨

fashion_mnist 데이터를 데이터로더에 전달

- 데이터로더 전달 이유: fashion_mnist 데이터를 메모리로 불러오기 위해

```
train_loader = torch.utils.data.DataLoader(train_dataset,
                                           batch_size=100)
test_loader = torch.utils.data.DataLoader(test_dataset,
                                           batch_size=100)
```

- torch.utils.data.DataLoader() → 원하는 크기의 배치 단위로 데이터 불러오기 or 순서가 무작위로 섞을 수 O(shuffle)
- 데이터로더에서 사용하는 파라미터
 - 첫 번째 파라미터: 데이터를 불러올 데이터셋 지정
 - batch_size: 데이터를 배치로 묶어줌(100개 단위로 데이터를 묶어서 불러옴)

분류에 사용될 클래스 정의

```
labels_map = {0 : 'T-Shirt', 1 : 'Trouser', 2 : 'Pullover',
              3 : 'Dress',
              4 : 'Coat', 5 : 'Sandal', 6 : 'Shirt', 7 : 'Sneaker',
              8 : 'Bag', 9 : 'Ankle Boot'}
fig = plt.figure(figsize=(8,8)); #출력할 이미지의 가로세로 길이 (단위 inch)
columns = 4;
rows = 5;
for i in range(1, columns*rows +1):
    img_xy = np.random.randint(len(train_dataset));
    img = train_dataset[img_xy][0][0,:,:]
    fig.add_subplot(rows, columns, i)
    plt.title(labels_map[train_dataset[img_xy][1]])
    plt.axis('off')
    plt.imshow(img, cmap='gray')
plt.show() #20개의 이미지 데이터를 시작적으로 표현
```

- np.random : 무작위로 데이터 생성할 때 사용
- np.random.randint() : 이산형 분포 갖는 데이터에서 무작위 표본 추출할 때 사용 (이산형: 값이 띄엄띄엄 떨어져 있는 형태 ↔ 연속형)
 - random.randint(len(train_dataset)) : 0 ~(train_dataset의 길이) 값 갖는 분포에서 랜덤한 숫자 한 개 생성

```
import numpy as np
np.random.randint(10) #0~10 임의의 숫자 출력
np.random.randint(1,10) #1~9 임의의 숫자 출력
```

- random.rand

```
np.random.rand(8) #0~1 사이의 정규표준분포 난수를 행렬로 (1x8) 출력
np.random.rand(4,2) #0~1 사이의 정규표준분포 난수를 행렬로 (4X2) 출력
```

- random.randn

```
np.random.randn(8)
#평균 0, 표준편차 1인 가우시안 정규분포 난수를 행렬로 (1X8) 출력
np.random.randn(4,2)
#평균 0, 표준편차 1인 가우시안 정규분포 난수를 행렬로 (4X2) 출력
```

- train_dataset 이용한 3차원 배열 생성

```
import numpy as np
examp = np.arange(0,100,3) #1~99숫자에서 3씩 건너뛴 행렬 생성
examp.resize(6,4) #행렬 크기 6x4로 조정
examp
examp[3] #3행
examp[3,3] #3행 3번째 열 요소
examp[3][3] #3행 3번째 열 요소
```

```
examp = np.arange(0,500,3)
examp.resize(3,5,5)
examp
examp[2][0][3]
```

- train_dataset[img_xy][0][0,:,:,]의미 : train_dataset에서 [img_xy][0][0,:,:,]에 해당하는 요소 값을 가져오겠다

분류에 사용될 클래스 정의

- 합성곱 신경망 vs 합성곱 신경망 아닌 심층 신경망
 - ConvNet 적용 X 네트워크 먼저 만들어보자

```
class FashionDNN(nn.Module):
    def __init__(self):
        super(FashionDNN, self).__init__()
        self.fc1 = nn.Linear(in_features=784, out_features=
256)

        self.drop = nn.Dropout(0.25)
        self.fc2 = nn.Linear(in_features=256, out_features=
128)

        self.fc3 = nn.Linear(in_features=128, out_features=
10)

    def forward(self, input_data):
        out = input_data.view(-1, 784)
        out = F.relu(self.fc1(out))
        out = self.drop(out)
        out = F.relu(self.fc2(out))
        out = self.fc3(out)
        return out
```

- 클래스 형태의 모델 - 항상 torch.nn.Module 상속받음
- __init__() - 객체가 갖는 속성 값 초기화. 객체 생성시 자동으로 호출됨
- super(FashionDNN, self).__init__() : FashionDNN이라는 부모(super)클래스 상속 받겠다
- nn : 딥러닝 모델(네트워크) 구성에 필요한 모듈이 모여 있는 패키지
- Linear - 단순 선형 회귀 모델 만들 때 사용
- nn.Linear에서 사용되는 파라미터
 - in_features: 입력의 크기(input size)
 - out_features: 출력의 크기(output size)
 - 실제 데이터 연산되는 forward() 에는 첫 번째 파라미터 값만 넘겨줌. 두 번째 파라미터에서 정의된 크기가 forward() 연산의 결과
- torch.nn.Dropout(p)

- p만큼의 비율로 텐서의 값 = 0
- 0 x 값들 - 기존 값에 $(1/(1-p))$ 만큼 곱해서 커짐
- forward() : 모델이 학습 데이터 입력받아 순전파(forward propagation) 학습 진행시킴
 - 반드시 forward 이름의 함수여야 함
 - 객체를 데이터와 함께 호출하면 자동으로 실행됨
 - 순전파 연산: $H(x)$ 식에 입력 x로부터 예측된 y를 얻는 것
- 파이토치에서의 뷰(view) - 넘파이의 reshape과 같은 역할. 텐서의 크기(shape) 변경
 - input_data.view(-1, 784) : input_data를 (?, 784) 의 크기로 변경하라. 2차원 텐서로 변경하되, (?, 784)의 크기로 변경해라
 - 첫 번째 차원(-1) : 사용자가 잘 모르겠으나 파이토치에게 맡기겠다
 - 두 번째 차원의 길이는 784를 가지도록 해라
- (활성화 함수 사용) nn vs nn.functional
 - nn

```
import torch
import torch.nn as nn
inputs = torch.randn(64, 3, 244, 244)
conv = nn.Conv2d(in_channels=3, out_channels=64, kernel_size=3, padding=1)
#세 개의 채널이 입력되어 64개의 채널이 출력되기 위한 연산. 3x3 크기의 커널 사용
layer = nn.Conv2d(1, 1, 3)
```

- nn.functional

```
import torch.nn.functional as F
inputs = torch.randn(64, 3, 244, 244)
weight = torch.randn(64, 3, 3, 3)
bias = torch.randn(64)
outputs = F.conv2d(inputs, weight, bias, padding=1)
```

- nn.Conv2d - input_channel과 output_channel 사용해서 연산
- functional.conv2d - 입력과 가중치 자체를 직접 넣어줌

(직접 넣어준다 = 가중치를 전달해야 할 때마다 가중치 값을 새로 정의해야함)

구분	nn.xx	nn.functional.xx
형태	nn.Conv2d: 클래스 nn.Module 클래스를 상속받아 사용	nn.functional.conv2d: 함수 def function (input)으로 정의된 순수한 함수
호출 방법	먼저 하이퍼파라미터를 전달한 후 함수 호출을 통해 데이터 전달	함수를 호출할 때 하이퍼파라미터, 데이터 전달
위치	nn.Sequential 내에 위치	nn.Sequential에 위치할 수 없음
파라미터	파라미터를 새로 정의할 필요 없음	가중치를 수동으로 전달해야 할 때마다 자체 가중치를 정의

객체

- 파이토치 근간 : C++ (→ 2장 파이토치의 아키텍처 참고) → C++에서 사용하는 객체 지향 프로그램 특징을 파이토치에서도 사용 = 객체
- 객체 지향 프로그래밍: 프로그래밍에서 필요한 데이터를 추상화하여 속성이나 행동, 동작, 특징 등을 객체로 만들고, 그 객체들이 서로 유기적으로 동작하도록 하는 프로그래밍 방법. 재사용성 ↑
- 객체(object): 메모리를 할당받아 프로그램에서 사용되는 모든 데이터. 변수, 함수 등등
 - 객체명 = 클래스명()

클래스와 함수

- 함수(function): 하나의 특정 작업 수행 위해 독립적으로 설계된 프로그램 코드
 - 함수의 호출: 특정 작업만 수행할 뿐 그 결과값 계속 사용하기 위해서는 반드시 어딘가에 따로 그 값 저장해야함
 - 함수 포함한 프로그램 코드 일부 재사용 위해서는 해당 함수 + 데이터가 저장되는 변수까지도 관리해야함

```
def add(num1, num2):
    result = num1 + num2
    return result
print(add(1,2))
print(add(2,3))
```

- 클래스(class): 함수 + 관련된 변수까지도 한꺼번에 묶어서 관리하고 재사용 가능하게 함

```

class Calc:
    def __init__(self): #객체 생성시 호출하면 실행되는 초기화 함수
        self.result = 0
    def add(self, num1, num2):
        self.result = num1 + num2
        return self.result
obj1 = Calc()
obj2 = Calc()
print(obj1.add(1,2))
print(obj1.add(2,3))
print('-----')
print(obj2.add(2,2))
print(obj2.add(2,3))

```

- 클래스의 두 개의 객체는 독립적으로 연산됨
- 개별적 함수로 구현했다면 복잡했을 코드가 간결해짐

심층 신경망에서 필요한 파라미터 정의

```

learning_rate = 0.001;
model = FashionDNN();
model.to(device)
criterion = nn.CrossEntropyLoss(); #분류 문제에서 사용하는 손실 함수
optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate);
print(model)

```

- 옵티마이저 위한 경사 하강법 - Adam 사용

심층 신경망 이용한 모델 학습

```

num_epochs = 5
count = 0
loss_list = []
iteration_list = []
accuracy_list = []

```

```

predictions_list = []
labels_list = []

for epoch in range(num_epochs):
    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)

        train = Variable(images.view(100, 1, 28, 28))
        labels = Variable(labels)

        outputs = model(train) #학습 데이터를 모델에 적용
        loss = criterion(outputs, labels)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        count += 1

    if not (count % 50): #count를 50으로 나누었을 때 나머지가 0이 아니라면 실행
        total = 0
        correct = 0
        for images, labels in test_loader:
            images, labels = images.to(device), labels.to(device)

            labels_list.append(labels)
            test = Variable(images.view(100, 1, 28, 28))
            outputs = model(test)
            predictions = torch.max(outputs, 1)[1].to(device)

            predictions_list.append(predictions)
            correct += (predictions == labels).sum()
            total += len(labels)

        accuracy = correct * 100 / total
        loss_list.append(loss.data)
        iteration_list.append(count)
        accuracy_list.append(accuracy)

```

```

    if not (count % 500):
        print("Iteration: {}, Loss: {}, Accuracy: {}".format(count, loss.data, accuracy))

```

- 배열, 행렬과 같은 리스트 사용 방법
 - 비어 있는 배열, 행렬 생성
 - append 메서드 이용해 데이터 하나씩 추가
- for 구문 사용하여 레코드(행, 가로줄) 하나씩 가져옴
 - for x, y in train: 처럼 in 앞에 변수 두 개 지정 → 레코드에서 요소 두 개 꺼내오겠다
- 데이터 처리 위해선 모델과 데이터가 동일한 장치(CPU, GPU)에 있어야 함
 - model.to(device)가 GPU 사용 → images.to(device), labels.to(device)도 GPU에서 처리되어야함
 - CPU에서 처리된 데이터를 GPU 모델에 적용하거나 반대 → 런타임 오류 발생
- Autograd: 자동 미분 수행하는 파이토치 패키지
 - 자동 미분에 대한 값 저장 위해 테이프(tape) 사용
 - 순전파 - 수행파는 모든 연산 저장
 - 역전파 - 저장된 값들 꺼내서 사용
 - Variable 사용 - 역전파 위한 미분 값 자동으로 계산(torch.autograd → Variable)
- 분류 문제에 대한 정확도는 전체 예측에 대한 정확한 예측의 비율로 표현 가능

```

classification accuracy = correct predictions / total predictions
classification accuracy = correct predcions / total predictions * 100
#결과에 100 곱하여 백분율로 표시
error rate = (1 - (correct predictions / total predictions)) * 100
#분류 문제에 대한 정확도 - 값 반전시켜 오분류율 or 오류율로 표현

```

- 분류 문제에서 클래스 세 개 이상일 경우 주의사항

- 정확도 80 % 이상일때, 하지만 80%라는 값이 모든 클래스가 동등하게 고려된 것인지, 특정 클래스의 분류가 높았던 것인지에 대해 알 수 없음 유의
- 정확도 90% 이상일때, 하지만 100개의 데이터 중 90개가 하나의 클래스에 속할 경우 90%의 정확도는 높다고 할 수 X
⇒ 모든 데이터를 특정 클래스에 속한다고 예측해도 90%의 예측 결과가 나오기 때문에 데이터 특성에 따라 정확도 잘 관측해야함

합성곱 네트워크 생성

```
class FashionCNN(nn.Module):
    def __init__(self):
        super(FashionCNN, self).__init__()
        self.layer1 = nn.Sequential(
            nn.Conv2d(in_channels=1, out_channels=32, kernel_size=3, padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2)
        )
        self.layer2 = nn.Sequential(
            nn.Conv2d(in_channels=32, out_channels=64, kernel_size=3),
            nn.BatchNorm2d(64),
            nn.ReLU(),
            nn.MaxPool2d(2)
        )
        self.fc1 = nn.Linear(in_features=64*6*6, out_features=600)
        self.drop = nn.Dropout2d(0.25)
        self.fc2 = nn.Linear(in_features=600, out_features=120)
        self.fc3 = nn.Linear(in_features=120, out_features=10)

    def forward(self, x):
        out = self.layer1(x)
        out = self.layer2(out)
        out = out.view(out.size(0), -1)
        out = self.fc1(out)
```

```

out = self.drop(out)
out = self.fc2(out)
out = self.fc3(out)
return out

```

- nn.Sequential - __init__()에서 사용할 네트워크 모델 정의 + forward() 함수에서 구현될 순전파를 계층 형태로 가독성 높은 코드로 작성 가능. 계층을 차례로 쌓을 수 있도록 $Wx + b$ 와 같은 수식과 활성화 함수를 연결해 주는 역할
⇒ 여러 계층을 하나의 컨테이너에 구현하는 방법
 - 데이터가 각 계층을 순차적으로 지나갈 때 사용하면 좋음
- 합성곱층 - 합성곱 연산 통해 이미지 특징 추출
 - 합성곱: 커널(필터)라는 $n \times m$ 행렬이 높이 x 너비 크기의 이미지를 처음부터 끝까지 훑으면서 각 원소끼리 곱한 후 모두 더한 값 출력

```

nn.Conv2d(in_channels=1, out_channels=32, kernel_size=3,
padding=1)

```

- 파라미터
 - in_channels: 입력 채널 수 - 흑백 이미지 1(w x h), RGB - 3(Red, Green, Blue)(w x h x c)
 - 채널: 2D 합성곱층에 이미지 적용 가정. 3차원으로 생각하면 채널 = 깊이
 - out_channels: 출력 채널 수
 - kernel_size: 커널 크기 = 필터
 - 커널: 이미지 특징 찾아내기 위한 공용 파라미터. CNN의 학습 대상은 필터 파라미터. 입력 데이터를 스트라이드 간격으로 순회하며 합성곱 계산
 - kernel_size = 3 → 커널 크기 (3,3) 정사각형/ 직사각형 (3,5)처럼 지정
 - padding: 패딩 크기. 출력 크기 조정 위해 입력 데이터 주위에 0을 채움. 패딩 값 ↑ → 출력 크기 ↑
- BatchNorm2d: 학습 과정에서 각 배치 단위별로 데이터가 다양한 분포를 가지더라도 평균과 분산을 이용하여 정규화하는 것
 - (그림)배치 단위나 계층에 따라 입력 값의 분포가 모두 다르지만 정규화를 통해 분포를 가우시안 형태로 만듦. 평균은 0, 표준편차는 1로 데이터의 분포 조정

- MaxPool2d - 이미지 크기 축소. 풀링 계층은 합성곱층의 출력 데이터를 입력으로 받아 출력 데이터(activation map)의 크기를 줄이거나 특정 데이터를 강조. 풀링 계층 처리 방법 - 최대 풀링(max pooling) & 평균 풀링(average pooling) & 최소 풀링(min pooling)
 - kernel_size: m x n 행렬로 구성된 가중치
 - stride: 입력 데이터에 커널(필터) 적용할 때 이동할 간격. 스트라이드 값 ↑ → 출력 크기 ↓
- 클래스 분류 위해선 이미지 형태의 데이터 → 배열 형태로 변환해야함. 이때 Conv2d에서 사용하는 하이퍼파라미터 값들에 따라 출력 크기 달라짐 = 패딩, 스트라이드 값에 따라. 이렇게 줄어든 출력 크기는 최종적으로 분류 담당하는 완전연결층으로 전달됨

```
nn.Linear(in_features=64*6*6, out_features=600)
```

- in_features: 입력 데이터 크기. 이전까지 수행했던 Conv2d, MaxPool2d - 이미지 데이터를 입력으로 받아 처리. But 그 출력 결과 완전연결층으로 보내기 전 1차원으로 변경해 주어야 함.

- Conv2d 계층에서 출력 크기 구하는 공식

- 출력 크기 = $(W - F + 2P) / S + 1$
 - W: 입력 데이터 크기(input_volume_size)
 - F: 커널 크기(kernel_size)
 - P: 패딩 크기(padding_size)
 - S: 스트라이드(strides)

```
nn.Conv2d(in_channels=1, out_channels=32, kernel_size=3, padding=1)
```

- $(784 - 3 + (2 * 1)) / 1 + 1 = 784$
 ⇒ fashion_mnist의 입력 데이터 크기 = 784, stride 기본값 (1,1)
 ⇒ 출력 형태 [32, 784, 784]

- MaxPool2d 계층에서 출력 크기 구하는 공식

- 출력 크기 = IF / F
 - IF: 입력 필터 크기(input_filter_size, 또한 바로 앞의 Conv2d의 출력 크기이기도 함)

- F: 커널 크기(kernel_size)

```
nn.MaxPool2d(kernel_size=2, stride=2)
```

- $784/2=392$
⇒ 출력 형태 [32, 392, 392]
- out_features: 출력 데이터 크기
- 합성곱층에서 완전연결층으로 변경되기 때문에 데이터 형태를 1차원으로 바꾸어 줌
 - out.size(0) ⇒ 100 (100, ?) 크기의 텐서로 변경 의미
 - out.view(out.size(0), -1) - -1: 행 수는 알고 있지만 열 수는 알지 못할 때

합성곱 네트워크 위한 파라미터 정의

```
learning_rate = 0.001;
model = FashionCNN();
model.to(device)
criterion = nn.CrossEntropyLoss();
optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate);
print(model)
```

모델 학습 및 성능 평가

```
num_epochs = 5
count = 0
loss_list = []
iteration_list = []
accuracy_list = []
predictions_list = []
labels_list = []
for epoch in range(num_epochs):
    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)

        train = Variable(images.view(100, 1, 28, 28))
        labels = Variable(labels)
```

```

        outputs = model(train)
        loss = criterion(outputs, labels)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        count += 1

    if not (count % 50):
        total = 0
        correct = 0
        for images, labels in test_loader:
            images, labels = images.to(device), labels.
to(device)

            labels_list.append(labels)
            test = Variable(images.view(100,1,28,28))
            outputs = model(test)
            predictions = torch.max(outputs, 1)[1].to(d
evice)

            predictions_list.append(predictions)
            correct += (predictions == labels).sum()
            total += len(labels)

        accuracy = correct * 100 / total
        loss_list.append(loss.data)
        iteration_list.append(count)
        accuracy_list.append(accuracy)

    if not (count % 500):
        print("Iteration: {}, Loss: {}, Accuracy: {}".
format(count, loss.data, accuracy))

```

- 심층 신경망과 비교

- 정확도가 약간 높음. 별 차이 없기 때문에 더 간편한 심층신경망만 사용해도 무난할 것 같음
- But 실제로 이미지 데이터 많아지면 → 단순 심층 신경망으로는 정확한 특성 추출 및 분류 불가능하므로 합성곱 신경망 생성할 수 있도록 학습해야함